University of Southampton

COMP6237 Data Mining
Lecture 7: Decision Trees

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

Lecturer (Assistant Professor) @ VLC of ECS

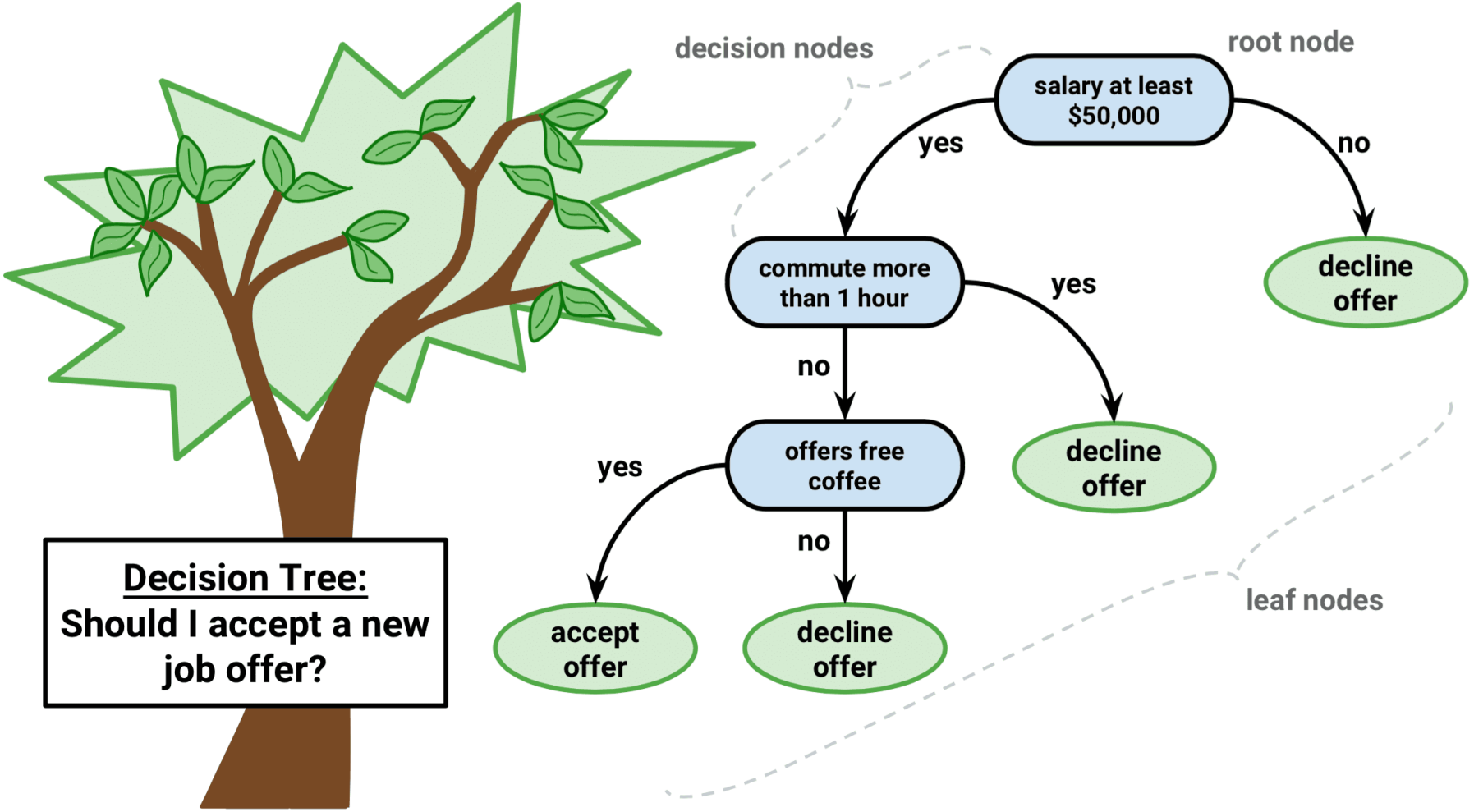
University of Southampton

Lecture slides available here:

http://comp6237.ecs.soton.ac.uk/zh.html

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

Decision Trees – Problem



Source: http://dataaspirant.com/2017/01/30/how-decision-tree-algorithm-works/

Decision Trees – Problem

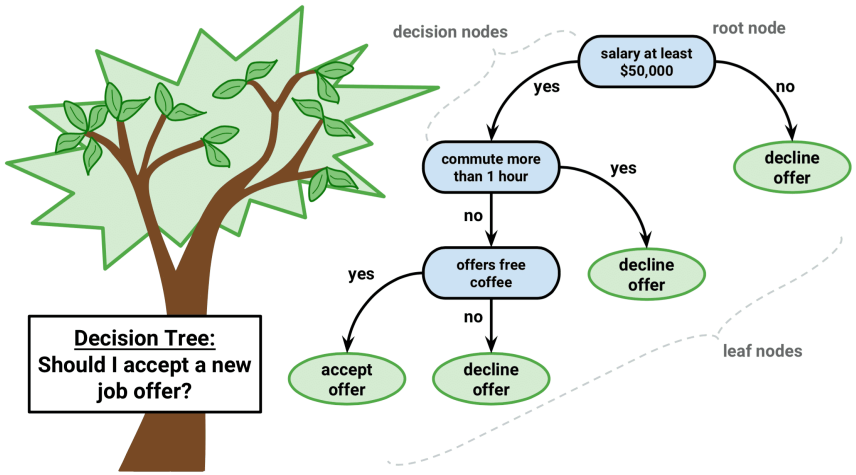
We encode an offer as features and a label:

- Salary (in £k): `salary_k`
- Commute time (hours): `commute_h`
- Free coffee? (0/1): `free_coffee`
- Label: `accept` (1 = accept, 0 = decline)

	salary_k	commute_h	free_coffee	accept
0	45	0.5	1	0
1	48	1.2	1	0
2	49	0.8	0	0
3	52	0.7	0	0
4	55	0.9	1	1
5	57	1.3	1	0
6	60	0.6	1	1

Toy "slide-style" rule (for reference):

> Accept **only if** salary ≥ 50k, commute ≤ 1 hour, and free coffee is offered.



Decision Trees – Approach: CART

The CART algorithm was published by Breiman *et al* in 1984

- Find best split for each feature - minimises impurity measure
- Find feature that minimises impurity the most
- Use the best split on that feature to split the node
- Do the same for each of the leaf nodes

The CART algorithm depends on an impurity measure. It uses Gini impurity

Gini impurity measures how often a randomly chosen element from a set would be incorrectly labelled if it was randomly labelled according to the distribution of labels in the set. The probabilities for each label are summed up.

Decision Trees – Approach: CART

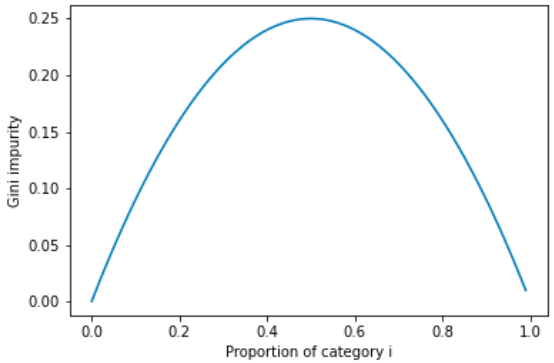
- Gini Index for a given node t :

The smaller the purer

$$GINI(t) = 1 - \sum_{j}^{n_c} [p(j|t)]^2$$

(where: n_c *is the number of classes, and p(j | t)* is the relative frequency of class j at node t).

  - Maximum: (1 - 1/n_c) when records are equally distributed among all classes, implying least interesting information
  - Minimum: (0.0) when all records belong to one single class, implying most interesting information



It reaches the minimum when all cases in the node fall into a single category

Decision Trees – Approach: CART

Before	decline	accept
	12	4

Candidate split A: salary≥50

	branch	n	decline(0)	accept(1)	gini
0	salary≥50	9	5	4	?
1	salary<50	7	7	0	?

Candidate split B: commute≤1h

	branch	n	decline(0)	accept(1)	gini
0	commute≤1h	13	9	4	?
1	commute>1h	3	3	0	?

Decision Trees – Approach: CART

$$GINI(t) = 1 - \sum_{j}^{n_c} [p(j|t)]^2$$

Before	decline	accept
	12	4

Gini (before) = 1- [p(d)^2+p(a)^2]
= 1- [(12/16)^2+(4/16)^2]
= 0.375

Candidate split A: salary≥50

	branch	n	decline(0)	accept(1)	gini
0	salary≥50	9	5	4	?
1	salary<50	7	7	0	?

Gini(s0) = 1- [p(d)^2+p(a)^2]
= 1- [(5/9)^2+(4/9)^2]
~ 0.494

Gini(s1) = 1- [p(d)^2+p(a)^2]
= 1- [(7/7)^2+(0/7)^2]
= 0

Candidate split B: commute≤1h

	branch	n	decline(0)	accept(1)	gini
0	commute≤1h	13	9	4	?
1	commute>1h	3	3	0	?

Gini(c0) = 1- [p(d)^2+p(a)^2]
= 1- [(9/13)^2+(4/13)^2]
~ 0.426

Gini(c1) = 1- [p(d)^2+p(a)^2]
= 1- [(3/3)^2+(0/3)^2]
= 1

Decision Trees – Approach: CART

- When a node p is split into k partitions (children), the quality of split is computed as,

$$GINI_{split} = \sum_{i=1}^{k} \frac{n_i}{n} GINI(i) \qquad (1)$$

where k is number of branches

$GINI(i)$ is the i -th branch-based GINI

n_i = number of records at child i,

n = number of records at node p.

Decision Trees – Approach: CART

$$GINI(t) = 1 - \sum_{j}^{n_c} [p(j|t)]^2$$

Before	decline	accept
	12	4

Gini (before) = 1- [p(d)^2+p(a)^2]
= 1- [(12/16)^2+(4/16)^2]
= 0.375

Candidate split A: salary≥50

	branch	n	decline(0)	accept(1)	gini
0	salary≥50	9	5	4	?
1	salary<50	7	7	0	?

Gini(s0) = 1- [p(d)^2+p(a)^2]
= 1- [(5/9)^2+(4/9)^2]
~ 0.494

Gini(s1) = 1- [p(d)^2+p(a)^2]
= 1- [(7/7)^2+(0/7)^2]
= 0

GINI_split = 0.2777777777777778 Gain = 0.0972222222222221

Candidate split B: commute≤1h

	branch	n	decline(0)	accept(1)	gini
0	commute≤1h	13	9	4	?
1	commute>1h	3	3	0	?

Gini(c0) = 1- [p(d)^2+p(a)^2]
= 1- [(9/13)^2+(4/13)^2]
~ 0.426

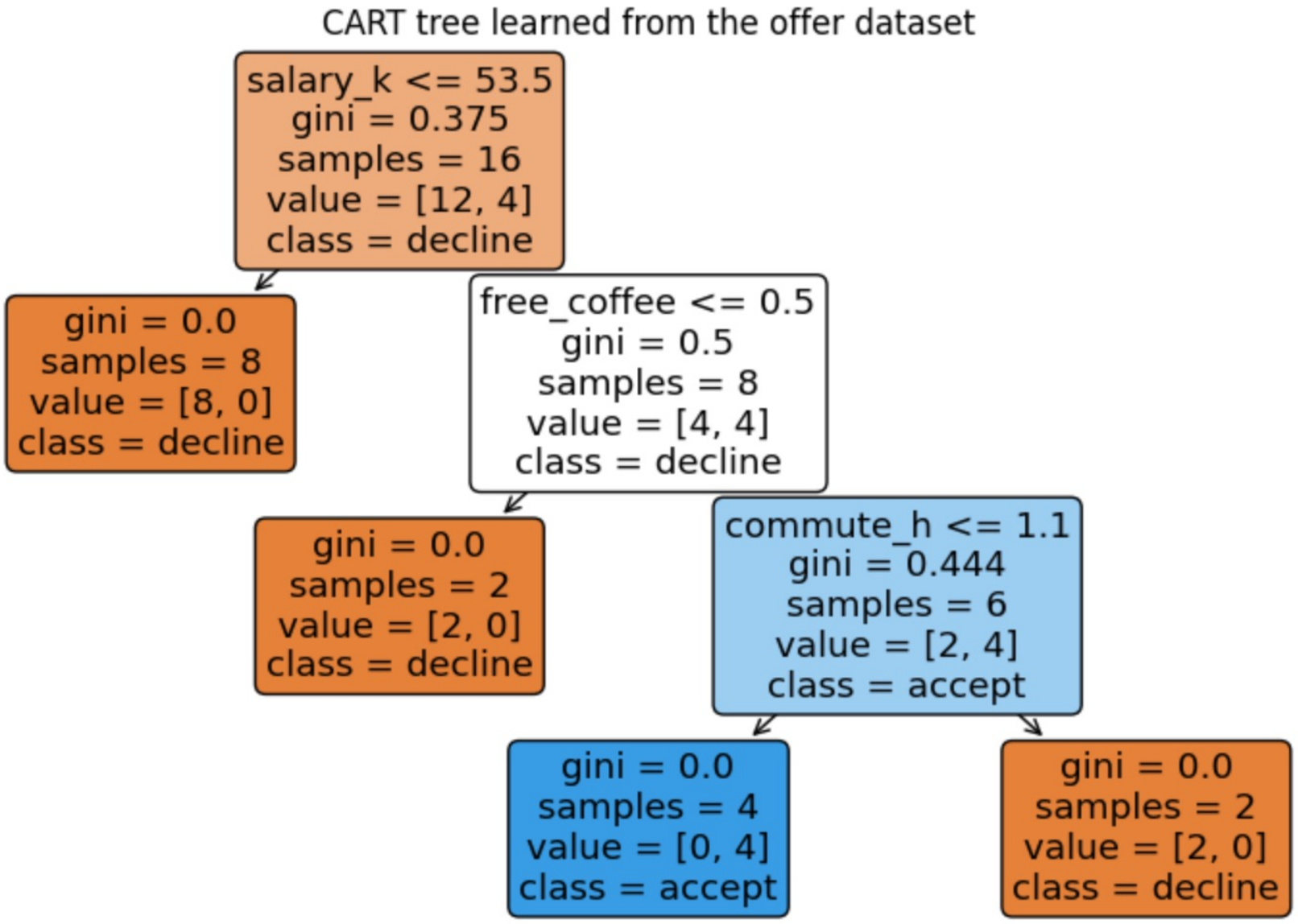
Gini(c1) = 1- [p(d)^2+p(a)^2]
= 1- [(3/3)^2+(0/3)^2]
= 1

GINI_split = 0.3461538461538462 Gain = 0.028

Better split (smaller GINI_split): salary≥50

$$GINI_{split} = \sum_{i=1}^{k} \frac{n_i}{n} GINI(i) \quad (1)$$

Decision Trees – Approach: CART



Demo: Decision-trees_Offer_Case

Decision Trees – Approach: ID3

Similar to CART, Iterative Dichotomy 3 (ID3) minimises entropy

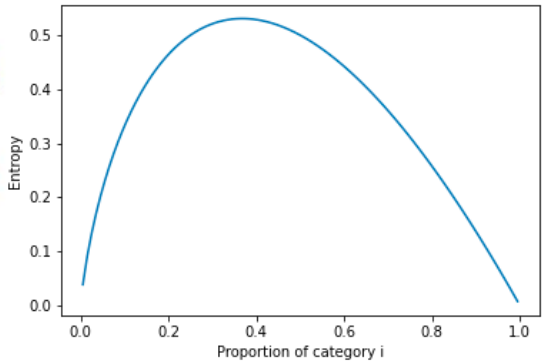
- Entropy at a given node t:

The smaller the purer

$$Entropy(t) = -\sum_{j=1}^{n_c} p(j|t) \log_2 p(j|t) \quad (1)$$

(where: n_c *is the number of classes, and p(j | t)* is the relative frequency of class j at node t).

  - Measures homogeneity of a node.
    - Maximum ($\log_2 n_c$) when records are equally distributed among all classes implying the **least** information
    - Minimum (0.0) when all records belong to one class, implying the **most** information
  - Entropy based computations are similar to the GINI index computations



Decision Trees – Approach: ID3

- Information Gain:

The higher the better

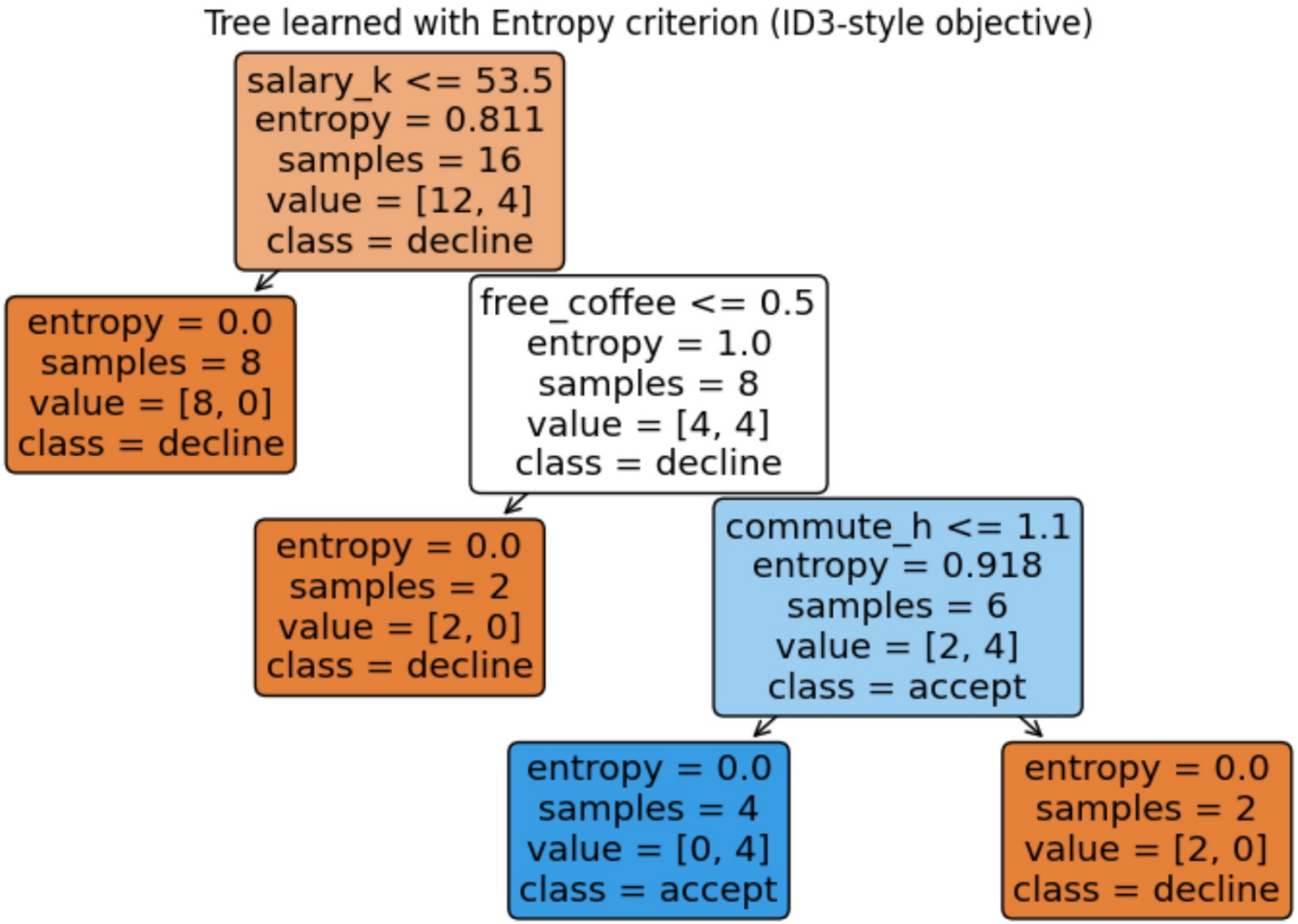
$$GAIN_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i)\right) \quad (1)$$

where $Entropy(p)$: entropy of Parent Node, $Entropy(i)$: entropy of the i-th branch/partition, n is split into k partitions; n_i is number of records in partition i

  - Measures Reduction in Entropy achieved because of the split. Choose the split that achieves most reduction (maximizes GAIN)
  - Used in ID3 and C4.5

Decision Trees – Approach: ID3

```
Entropy(parent) = 0.8112781244591328
E File display )ain (salary≥50)  = 0.2537978408001328
Entropy gain (commute≤1h) = 0.0877536667807961
Better split (higher gain): salary≥50
```



Demo: Decision-trees_Offer_Case

Decision Trees – Overfitting Problem

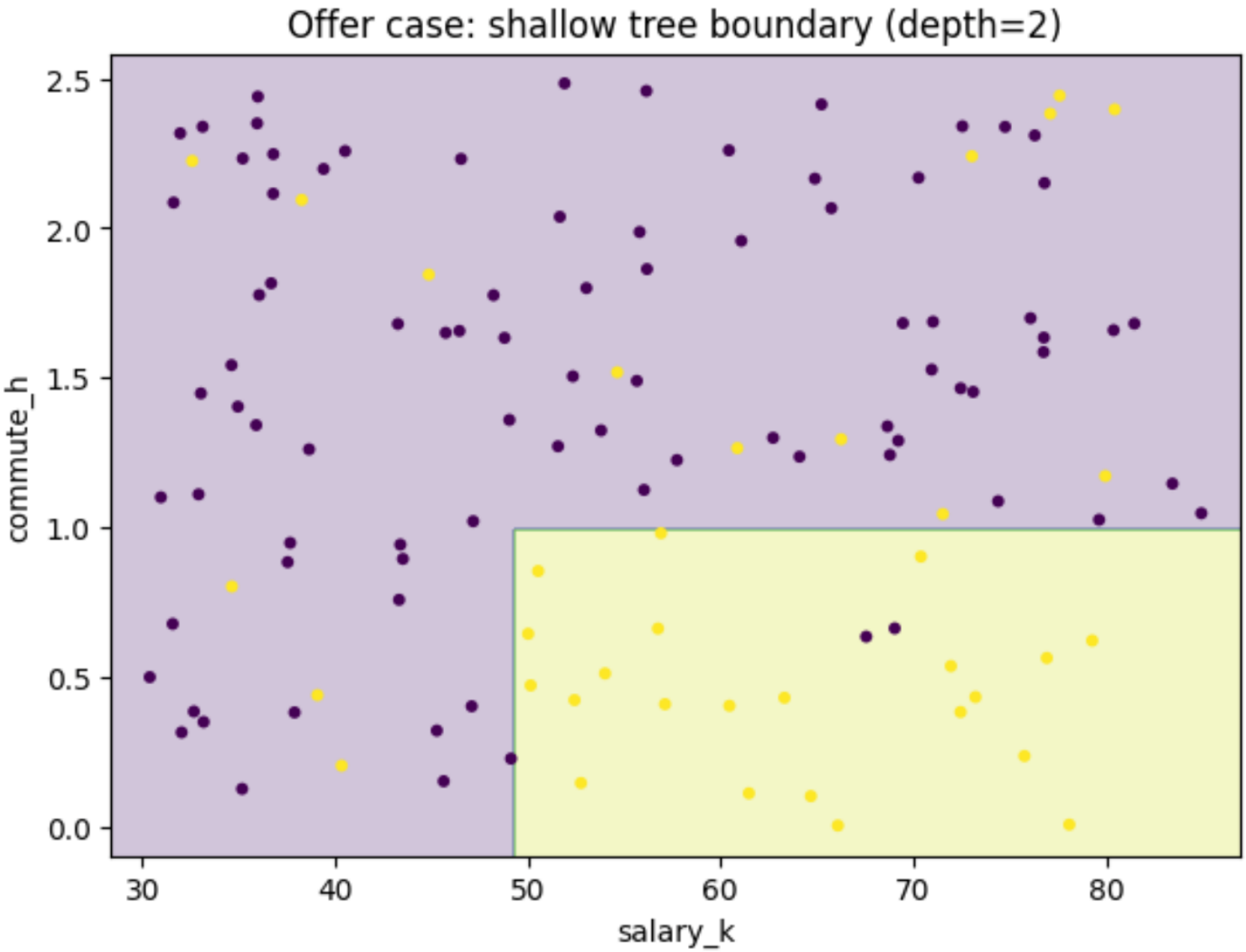
We generate a larger **offer-themed** dataset with label noise and irrelevant features, then compare:

- a shallow tree (regularised)
- a deep tree (can overfit)

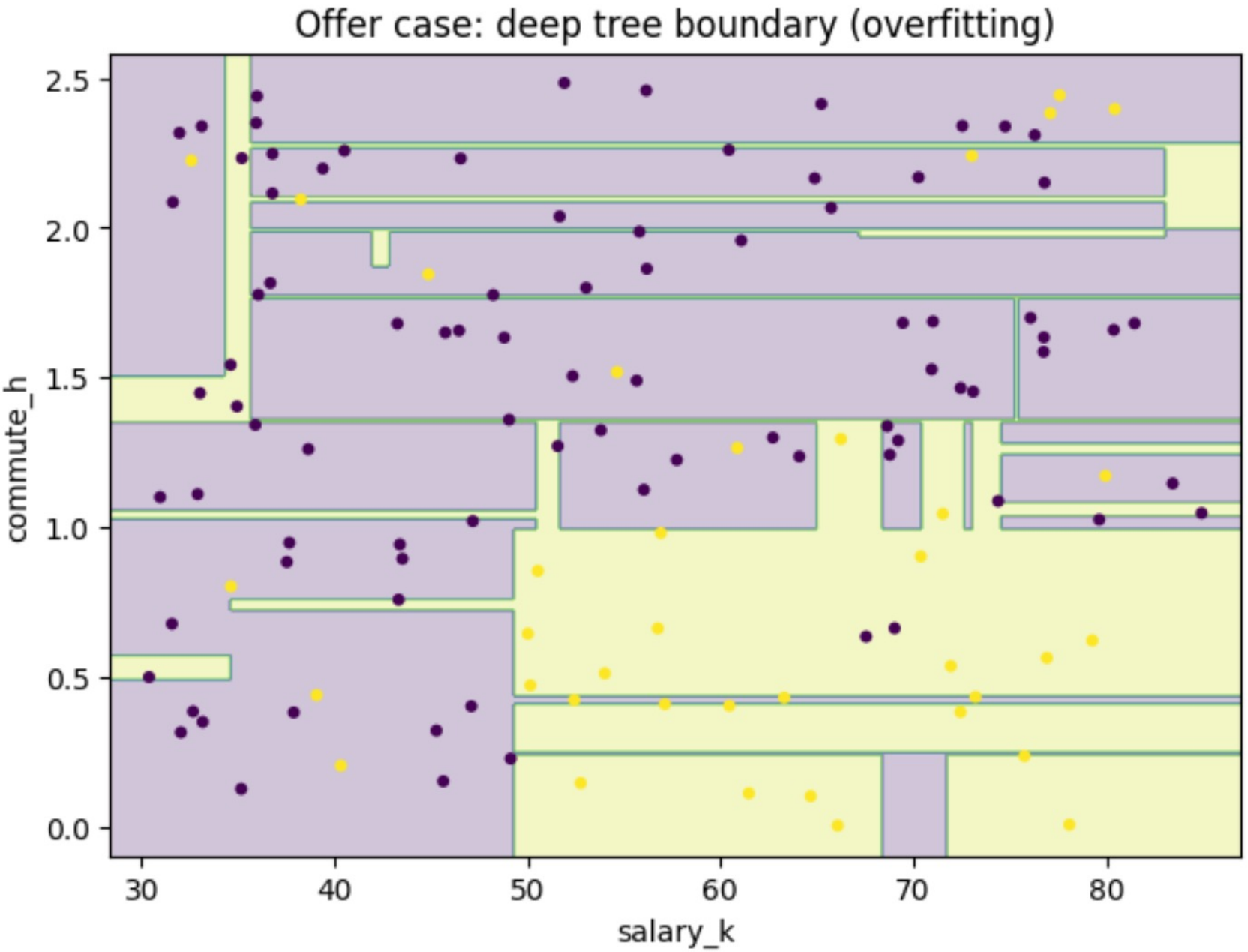
We also visualise a 2D boundary (salary vs commute) to make overfitting intuitive.

	salary_k	commute_h	free_coffee	bonus_k	team_size	office_plants	snack_score	accept
0	72.567583	1.846451	1	12.640980	12	13	0.446461	0
1	54.138314	0.792883	1	6.695842	29	19	-0.060092	1
2	77.222886	2.225049	1	14.294627	3	7	0.012579	0
3	68.355242	1.484576	0	9.761919	16	23	0.656819	0
4	35.179754	0.315066	0	1.738483	19	20	-0.727368	0

Decision Trees – Overfitting Problem

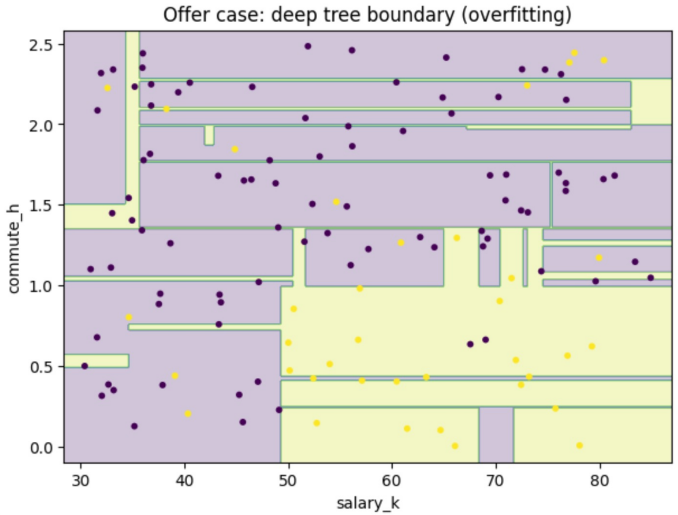


Decision Trees – Overfitting Problem



Decision Trees – Pruning Approach

Overfitting is a serious problem with Decision Trees



.. Are these splits really required here?

The trees it creates are too complex.

One solution is **pruning**

This can be done in a variety of ways, including:

- Reduced Error Pruning
- Entropy Based Merging

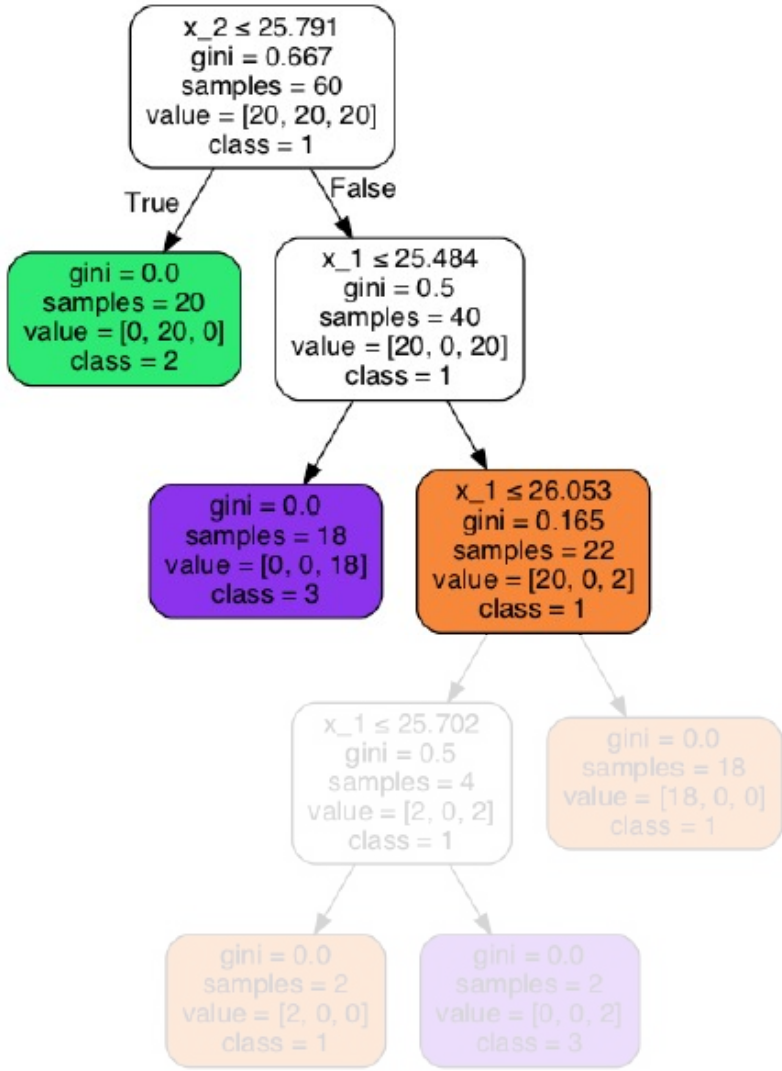
Decision Trees – Pruning Approach
Reduced Error Pruning

University of Southampton

Growing a decision tree fully, then removing branches without reducing predictive accuracy, measured using a *validation set*.

- Start at leaf nodes
- Look up branches at last decision split
- replace with a leaf node predicting the majority class
- If validation set classification accuracy is not affected, then keep the change

This is a simple and fast algorithm that can simplify over complex decision trees



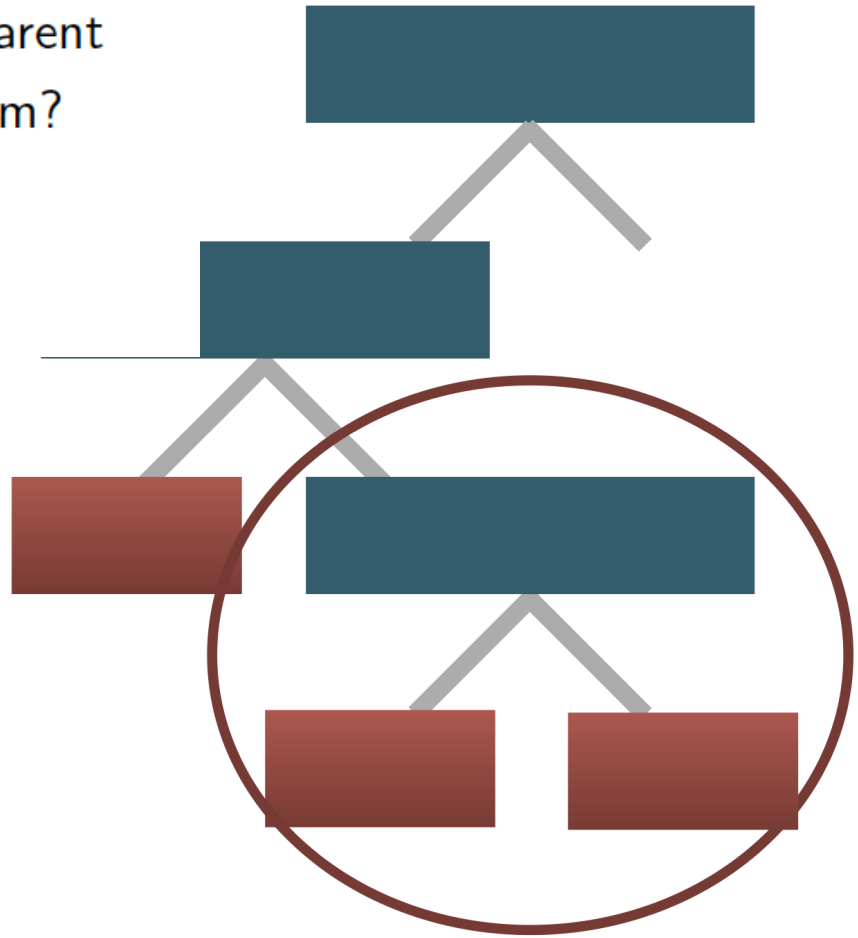
Decision Trees – Pruning Approach
Entropy Based Pruning

University of Southampton

Grow a decision tree fully, then

- Chose a pair of leaf nodes with the same parent
- What is the entropy gain from merging them?
- If lower than a threshold, merge nodes

This doesn't require additional data



Demo: Decision-trees_Offer_Case

Decision Trees – Ensemble methods

Bagging: Bootstrap aggregating
Uniformly sample initial dataset with replacement in to m subsets
For example:
if data set has 5 samples, $(s_1, s_2, s_3, s_4, s_5)$
make a whole bunch of similar data sets:

- $(s_5, s_2, s_2, s_1, s_5)$
- $(s_4, s_2, s_1, s_3, s_3)$
- $(s_2, s_5, s_3, s_1, s_1)$
- $(s_3, s_1, s_2, s_4, s_4)$

Train a different decision tree on each set
To classify, apply each classifier and chose the correct one by majority vote

Decision Trees – Ensemble methods

Boosting - Kearns and Valiant (1988):
"Can a set of weak learners create a single strong learner?"

We make a weighted sum of very weak learners
- so long as they all learn different things then it works!
AdaBoost:

- Train a weak learner on one sampled data set
- See what it does well on
- Weight the difficult data samples more
- repeat

This makes a series of weak learners that have learned how to use given features to classify easy and hard samples more correctly

Decision Trees – Ensemble methods

Gradient boosting with trees - Friedman (1999):
Generalise Adaboost to Gradient boosting to handle any loss function
Shortcomings are where the residuals are larger
So fit a tree to the residuals:

- $x_1, y_q - f(x_1)$
- $x_2, y_q - f(x_1)$
- $x_3, y_q - f(x_1)$
- $\vdots$
- $x_n, y_q - f(x_n)$

more detail available at

http://www.chengli.io/tutorials/gradient_boosting.pdf

Decision Trees – Ensemble methods

University of Southampton

Random Forests

Apply bagging
but when learning the tree for each subset, chose the split by searching over a random sample of the features

Reduces overfitting

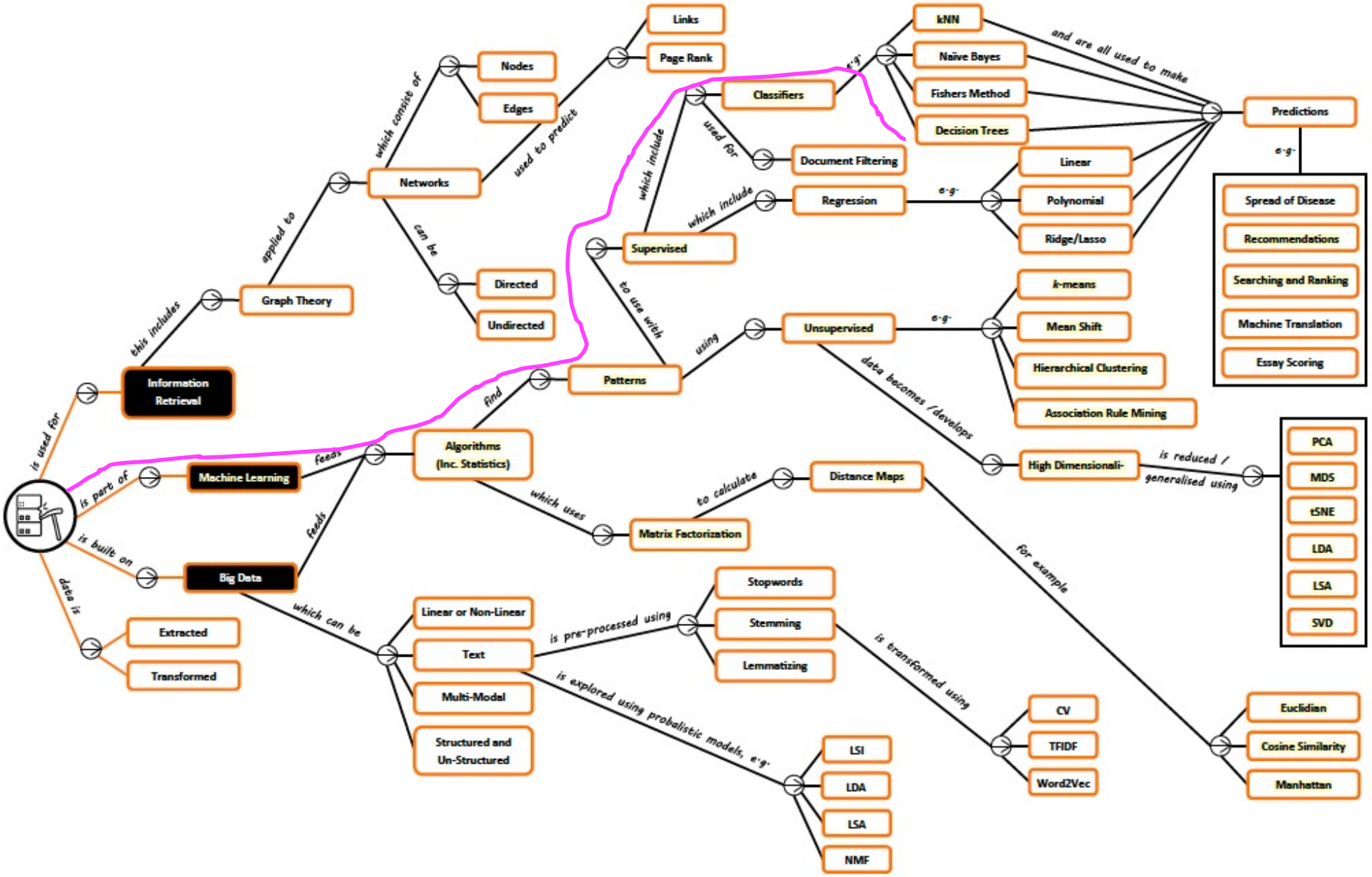
	model	test_acc
2	Random forest	0.875000
1	Bagging	0.854167
4	Gradient boosting	0.854167
3	AdaBoost (stumps)	0.829167
0	Single tree	0.808333

Demo: Decision-trees_Offer_Case

Decision Trees – Summary

- **CART** minimises impurity (e.g., Gini) to choose splits.
- **ID3/C4.5** maximise information gain (entropy reduction).
- Trees can **overfit**; pruning reduces complexity.
- **Ensembles** (bagging, random forests, boosting) usually improve generalisation.

Decision Trees – **Roadmap**



Decision Trees – **Textbook**

PART FOUR CLASSIFICATION

The classification task is to predict the label or class for a given unlabeled point. Formally, a classifier is a model or function M that predicts the class label $\hat{y}$ for a given input example $\mathbf{x}$, that is, $\hat{y} = M(\mathbf{x})$, where $\hat{y} \in \{c_1, c_2, \ldots, c_k\}$ and each c_i is a class label (a categorical attribute value). Classification is a *supervised learning* approach,

- Data Mining and Machine Learning: Fundamental Concepts and Algorithms M. L. Zaki and W. Meira

4 Classification: Alternative Techniques

The previous chapter introduced the classification problem and presented a technique known as the decision tree classifier. Issues such as model overfitting and model evaluation were also discussed. This chapter presents alternative techniques for building classification models—from simple techniques such as rule-based and nearest neighbor classifiers to more sophisticated techniques such as artificial neural networks, deep learning, support vector machines, and ensemble methods. Other practical issues such as the class imbalance and multiclass problems are also discussed at the end of the chapter.

- Introduction to Data Mining, *P. Tan et al*

Decision Trees – **Learning Outcomes**

- **LO1**: Demonstrate an understanding of decision tree fundamentals, such as (exam)
  - Calculating impurity and constructing a decision tree using algorithms like ID3, C4.5, etc, given a dataset and distance metric
  - Addressing overfitting in decision trees like pruning
  - Understanding ensemble methods using decision trees like bagging, boosting, random forests
- **LO2**: **Implement** the learned algorithms using decision trees (course work)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf*
- *ChatGPT or other AI-based techs*